

Tomo

Read, write, and connect.

Background

Tomo serves a large global community of fanfiction readers and writers who are highly active but lack a centralized modern platform. Most rely on a small number of existing platforms that are often outdated, poorly optimized for mobile, or no longer actively maintained. As a result, the community feels “homeless” despite its size and engagement. These users prioritize creative freedom and seek a platform designed specifically for fanfiction that supports long-form reading, detailed tagging systems, and active community discussions.

Problem Statement

Although the fanfiction community continues to grow, the digital platforms supporting it have not kept pace. Users face several key challenges including limited platform choices, outdated or abandoned apps, and poor mobile experiences due to a lack of app-first design. Features that modern users expect such as offline reading, real-time interaction, and seamless syncing are often missing. In addition, the social aspect of fandom is fragmented with discussions happening on separate platforms, making it harder for readers and writers to connect.

Solution

Tomo addresses these gaps by providing a modern all-in-one mobile application where users can read, write, and interact within a single high-performance ecosystem tailored to the needs of the fanfiction community.

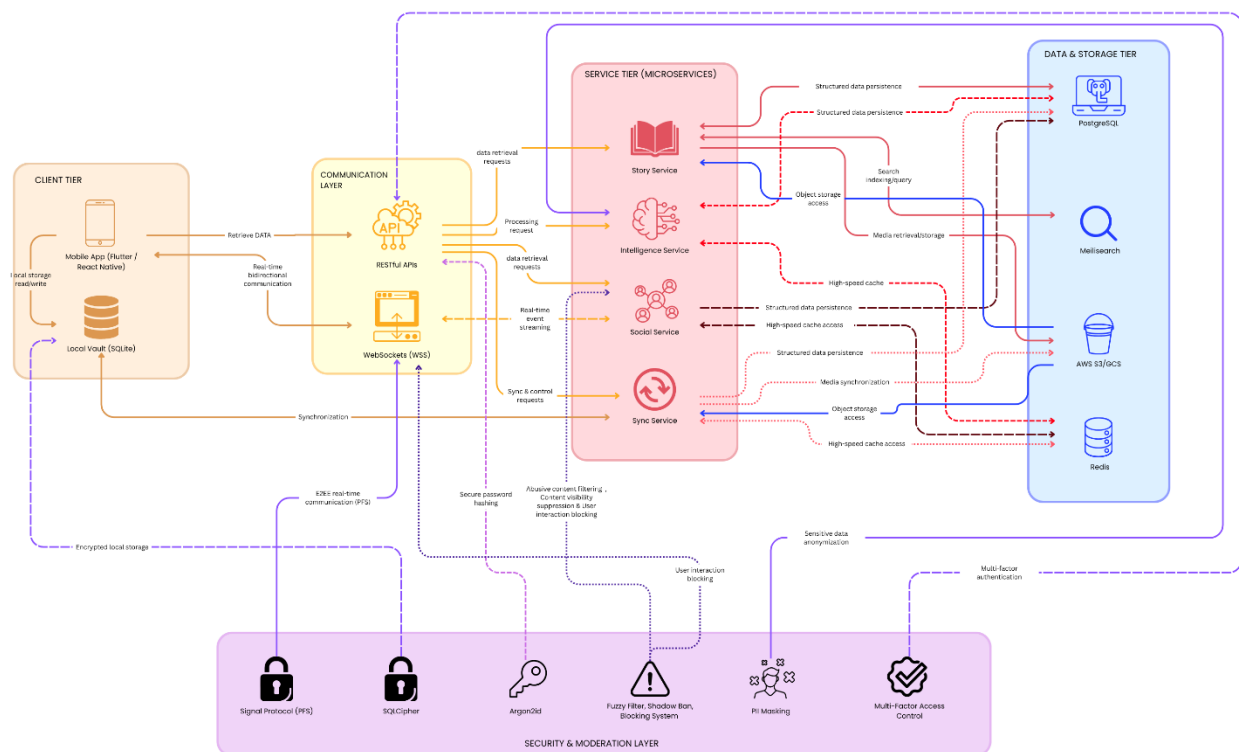
Architectural Design

i. Chosen Architectural Pattern

Tomo adopts a **Cloud-Native Hybrid Architecture** that integrates Microservices with an Offline-First Client-Server model.

- **Microservices:** The backend is divided into independent services (Story, Social, Intelligence, Sync), enabling scalability, flexibility, and improved fault tolerance.
- **Offline-First:** Unlike traditional web-based archives, the system treats the mobile device as the primary workspace. Local data storage ensures the application remains fully functional even without internet connectivity.

ii. System Architecture Diagram



iii. Description of Components/Modules

Component	Description
Client Tier (Mobile App)	The front-end application built using a cross-platform framework. It includes the Local Vault (SQLite/Room) , which serves as the main data source during offline use.
Communication Layer	A dual-protocol gateway that uses REST APIs for standard operations (e.g., user profiles, story metadata) and WebSockets (WSS) for real-time features such as “Circles” and “Live-React.”
Service Tier (Microservices)	The core logic layer composed of: • Story Service: Handles Markdown rendering and archive indexing. • Social Service: Manages forums, private circles, and community moderation features. • Intelligence Service: Applies “Library DNA” logic to generate personalized recommendations. • Sync Service: Manages data synchronization between the device and the cloud..
Data & Storage Tier	A multi-database setup using PostgreSQL for structured data (e.g., users, tags), Redis for caching sessions, Meilisearch for fast full-text archive search, and AWS S3/GCS for storing large assets such as images and covers.
Security & Moderation	A dedicated layer that ensures system security, including Signal Protocol (end-to-end encryption) for private messaging, Argon2id for password protection, and SQLCipher for encrypting local databases. And also Includes moderation and access control mechanisms such as Fuzzy Filtering, Shadow Banning, Blocking, PII Masking, and Multi-Factor Authentication.

iv. Component Interaction and Data Flow

The system follows a **Local-State-Sync** approach to maintain the “Tomo Advantage”:

1. Request and Retrieval:

When a user searches for a story, the Client Tier sends a request through the API

Gateway. The Story Service retrieves data from Meilisearch (for fast search) and PostgreSQL (for detailed information), then returns the results to the user.

2. State Persistence (Offline Mode):

When a user saves (“Vaults”) a story, the Client Tier stores it in the device’s encrypted Local Vault (SQLite). Reading progress and user actions are saved locally first, after which the Sync Service synchronizes this data with the cloud.

3. Synchronization Flow:

When the device reconnects to the internet, the Sync Service synchronizes local and cloud data. It resolves any conflicts (such as updates made on multiple devices) and ensures the central cloud database remains accurate.

4. Intelligence Loop:

User activities (e.g., likes, reading history) is processed by the Intelligence Service after passing through a PII masking step. Before analysis, data passes through a PII masking step in the Security Layer to protect user privacy while generating personalized recommendations (Active Pulse feed).

5. Real-Time Social Interaction:

For features like “Buddy Reading” and “Circles,” WebSocket maintain a continuous connection. Messages are encrypted using the Signal Protocol before leaving the sender’s device and are decrypted only on the recipient’s device.

Technology Stack

CATEGORY	RECOMMENDED TECHNOLOGY
Frontend (mobile)	Flutter (Dart) or React Native
Backend (microservices)	Node.js (Express/NestJS) or Go (Golang)
Primary database	PostgreSQL
Caching & real-time	Redis
Search engine	Meilisearch
Local storage (the vault)	SQLite with SQLCipher
Cloud storage	AWS S3 or Google Cloud Storage
Real-time Communication	WebSockets (WSS)
Encryption	Signal Protocol

II. Justification of Technologies

1. Frontend: Flutter / React Native

Both Flutter and React Native support a single codebase that can be deployed across Android and iOS platforms, making them efficient choices for cross-platform mobile development. These frameworks also provide strong tools for building responsive and visually consistent user interfaces.

Justification:

- Enables cross-platform accessibility, allowing Tomo to reach users on different devices.
- Supports highly customizable UI, especially useful for features like the Adaptive Theme Engine (Dark, Sepia, OLED Black).
- Provides smooth performance for long-form reading and animations.

2. Backend: Node.js or Go

Node.js and Go are both designed to handle high levels of concurrency and are well-suited for microservices-based architectures. They are commonly used in scalable systems that require efficient handling of multiple simultaneous requests.

Justification:

- Node.js efficiently manages real-time communication through WebSockets for features like “Circles” and “Live-React.”
- Go offers strong performance and low memory usage, making it suitable for processing logic such as the “Library DNA” system.

3. Databases: PostgreSQL & Redis

PostgreSQL is a robust relational database capable of managing complex data relationships, while Redis is an in-memory data store designed for speed and quick data access. Together, they provide both reliability and performance.

Justification:

- PostgreSQL effectively handles complex relationships between stories, tags, and authors.

- Redis improves performance by caching session data and frequently accessed content like “Active Pulse” feeds.
- Enhances overall system responsiveness and reduces load times.

4. Search: Meilisearch

Meilisearch is a fast, user-friendly search engine that supports typo tolerance and advanced filtering, making it ideal for applications with large amounts of content.

Justification:

- Allows advanced filtering options such as including or excluding tags.
- Delivers fast search results, even with large datasets.
- Improves overall user experience by making content easier to find.

5. Local Persistence: SQLite + SQLCipher

SQLite is a lightweight and widely used local database for mobile applications, while SQLCipher adds encryption capabilities to secure stored data. This combination is effective for offline-first applications.

Justification:

- Enables reliable offline access through local data storage (“The Vault”).
- Ensures data security by encrypting locally stored content.
- Protects user data from unauthorized access.

6. Security: Signal Protocol

The Signal Protocol is a widely trusted standard for implementing end-to-end encryption in communication systems, ensuring that messages remain private between users.

Justification:

- Provides strong end-to-end encryption for private messaging features like “Circles.”
- Ensures that only intended recipients can access messages.
- Strengthens user trust by maintaining high privacy standards.

Architecture Evaluation

a. Support for Key Quality Attributes

1. Scalability

- **Horizontal Scaling:**
The use of Microservices (Story, Social, Intelligence, Sync) allows each service to scale independently. For instance, if a particular story becomes highly popular, the Story Service can be replicated across multiple servers to handle increased demand without impacting other services such as the Intelligence Service.
- **Asynchronous Processing:**
By leveraging Redis for caching and WebSockets for real-time communication, the system reduces bottlenecks. This enables thousands of users to participate in features like “Circles” simultaneously without degrading overall system performance.

2. Maintainability

- **Decoupled Modules:**
The layered architecture separates concerns effectively, ensuring that updates to the frontend (UI) do not require changes to backend logic or database structures.
- **Fault Isolation:**
Since services operate independently, failures in one module (e.g., the Social Hub) do not affect others (e.g., Reader Mode). This allows developers to perform maintenance or updates on individual services without disrupting the entire system.

3. Security

- **Encryption at Rest and in Transit:**
Local data is secured using SQLCipher, while user credentials in the cloud are protected through Argon2id hashing.
- **End-to-End Encryption (E2EE):**
The use of the Signal Protocol for private “Circles” ensures that only intended users can access messages, reinforcing strong privacy protection.
- **Privacy Controls:**
A PII masking layer enables the Intelligence Service to analyze user behavior for recommendations without exposing personally identifiable information.

b. Possible Limitations and Risks

1. Data Consistency (Sync Conflicts)

- **Risk:**
In an Offline-First system, a user may edit the same content on multiple devices (e.g., offline on a phone and online on a tablet).
- **Limitation:**
This can lead to version conflicts. The Sync Service must implement advanced conflict resolution strategies (such as Last-Write-Wins or vector clocks) to prevent data loss and maintain consistency.

2. Infrastructure Complexity and Cost

- **Risk:**
Operating multiple microservices alongside PostgreSQL, Redis, and a search engine increases system complexity and operational cost compared to a monolithic design.
- **Limitation:**
For smaller teams or academic projects, the required DevOps effort to maintain and coordinate these components can be significant.

3. Latency in “Library DNA” Processing

- **Risk:**
Generating personalized recommendations based on complex tag relationships requires substantial computation.
- **Limitation:**
If the Intelligence Service is not well-optimized, users may experience delays or outdated recommendations on the homepage.

4. Client-Side Resource Consumption

- **Risk:**
Extensive use of local storage (SQLite) and encryption (SQLCipher) may increase battery usage and storage demands on mobile devices.
- **Limitation:**
Users with older devices or limited storage capacity may encounter performance issues unless the application efficiently manages local data and caching.